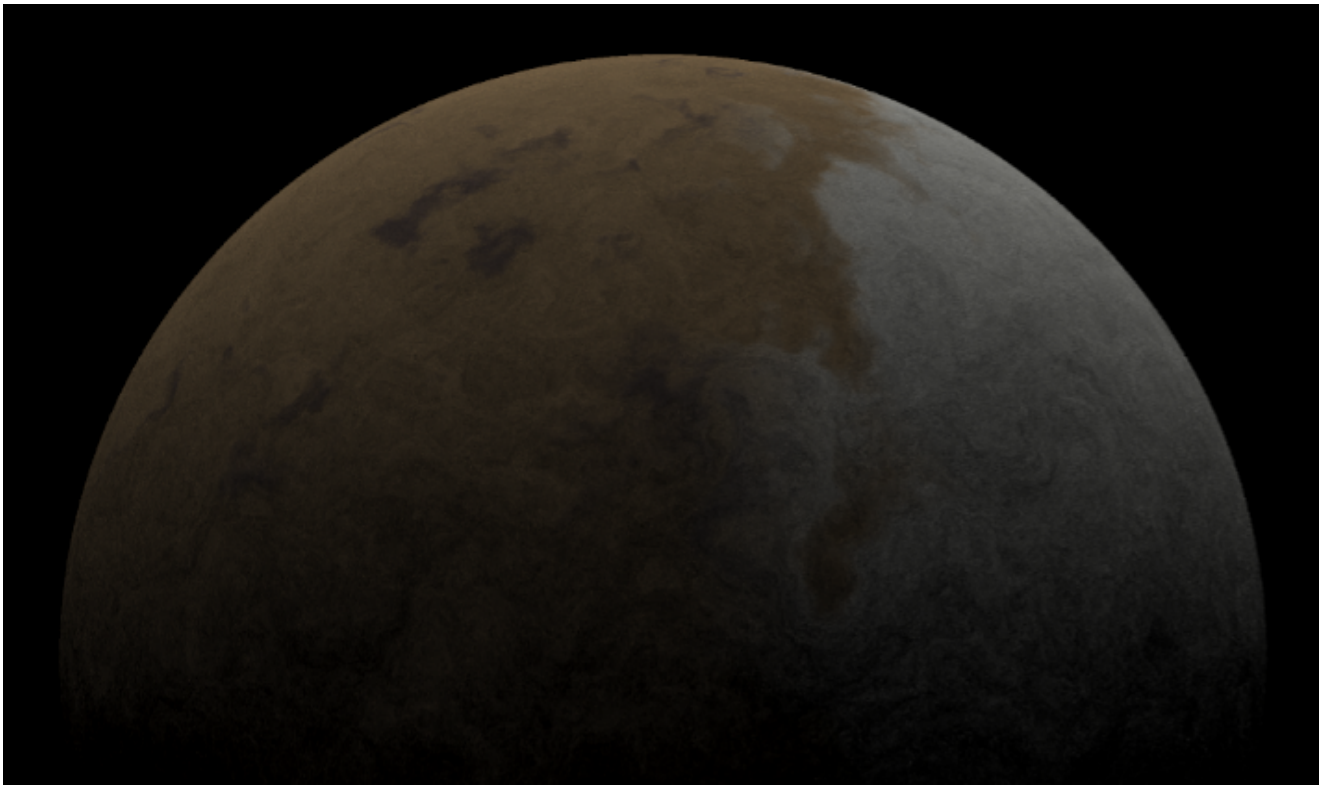
# DH2323 Computer Graphics and Interaction
# Real-Time Procedural Planet Renderer

Harald Tröjbom

KTH Royal Institute of Technology



## 1 Abstract

This project explores the implementation of a real-time procedural planet renderer using GPU compute shaders in Unity. The renderer combines ray-traced rendering with procedural terrain generation, and terrain height is represented with normals generated using analytical noise gradients. Surface parameters and masks are then used to generate terrain coloring and climate-dependent snow coverage.

The report describes the mathematical and graphical techniques used to generate the planet, including ray-sphere intersection, procedural terrain generation, terrain shading and camera controls. The visual results demonstrate that relatively simple procedural techniques can be combined to create visually convincing planets rendered in real time.

## 2 Introduction

Rendering realistic planets in real time is a difficult problem in computer graphics due to the large scale difference between a planet as a whole and its terrain features. The usual approach of using polygon meshes becomes increasingly difficult on a planetary scale, requiring large amounts of geometry and level-of-detail systems in order to represent the surface. Procedural generation instead allows terrain detail to be generated mathematically during rendering, reducing geometry while enabling potentially infinite detail.

Modern GPUs are suitable for this type of rendering due to their ability run many computations in parallel. This allows procedural generation and ray-based rendering calculations to be performed independently for large amounts of pixels simultaneously.

This project explores the implementation of a real-time procedural planet renderer using GPU compute shaders in Unity. The renderer combines ray-traced sphere rendering with procedural terrain generation based on

gradient noise, fractal Brownian motion and domain warping. Surface normals generated from analytical noise gradients are used to create the appearance of terrain without displacing the actual surface geometry. Additional procedural systems are then used to generate terrain coloring and climate-dependent snow coverage.

The goal of the project is to investigate how relatively simple procedural techniques can be combined to produce visually convincing planetary rendering in real time.

# 3 Related Work

Procedural generation is a widely used technique in computer graphics, especially for rendering large-scale environments where storing geometry becomes impractical. By generating terrain mathematically from noise functions, detailed surfaces can instead be generated dynamically during rendering. A notable example within the games industry is the game *No Man's Sky*, which relies heavily on procedural generation to populate an entire galaxy with unique large-scale planets [1].

This project builds upon previous work in this course, as well as articles written by graphics programmer Inigo Quilez [2][3][4]. His work on gradient noise, fractal Brownian motion and domain warping has laid an integral foundation for the procedural generation used when rendering the planet.

The ray-tracing setup used in this project builds upon techniques introduced in the rendering track of the DH2323 Computer Graphics and Interaction course on KTH, particularly ray-object intersection and direct illumination models [5].

# 4 Implementation

This section will go through the implementation of each technique that make up the planet renderer.

## 4.1 Shader Setup

The planet renderer is set up in Unity using a compute shader. Unity was chosen mainly as a framework, handling the window, input, and displaying the final image each frame. The image itself is synthesized inside of the compute shader, completely independent of Unity's built-in rendering pipeline. The shader writes directly to a RenderTexture that is displayed on the screen, where each shader thread computes the color of one pixel. The remainder of this section is devoted to determining the color of these pixels.

## 4.2 Ray-Traced Rendering

The color of the pixels on the screen is determined through ray-tracing. The ray-tracing setup described in this subsection is mostly based on one taught in the course this project is a part of [5]. For every pixel, the camera casts a ray into the scene. The direction of the ray $\mathbf{d}$ is calculated using the forward, right and up directions of the camera, $\mathbf{c}$, as basis vectors. They are scaled with the focal length $f$ and the centered pixel coordinates and the result is normalized:

$$\mathbf{d} = \text{normalize}\left(f\mathbf{c}_{forward} + u_x\mathbf{c}_{right} + u_y\mathbf{c}_{up}\right)$$

where $u_x$ and $u_y$ are calculated as

$$\mathbf{u} = (x - \frac{W}{2}, y - \frac{H}{2})$$

where x and y are the pixel screen coordinates and W and H are the pixel width and height of the screen. The halves of the width and height are subtracted to center the pixel coordinate around the middle of the screen.

Now that rays are cast from the camera, they need something to intersect. The planet geometry is represented by an implicit sphere, instead of the usual polygon mesh. This was chosen since ray-sphere intersections can be computed analytically which is generally faster. The sphere intersection is found by inserting the ray equation

$$\mathbf{p}(t) = \mathbf{o} + t\mathbf{d}$$

where $\mathbf{o}$ is the ray origin, $\mathbf{d}$ its direction and $t$ is the distance along the ray, into the implicit sphere equation

$$||\mathbf{p} - \mathbf{c}||^2 = r^2$$

where $\mathbf{p}$ is a point along the ray, $\mathbf{c}$ is the center position of the sphere and $r$ is its radius. This produces a quadratic equation whose solutions are intersection points between the ray and the sphere. If no real solution exists, the ray misses the planet and the pixel is colored black. Otherwise, the closest intersection distance is selected and used to compute the visible surface point. The sphere normal is then calculated from the normalized direction between the planet center and the intersection point.

The planet is then illuminated using direct lighting. Since a planet is usually very far away from its sun, the rays of light that reaches its surface are almost parallel. For this reason, it is enough to represent the sun using just the direction towards it, which is then the same for all points on the planet. The illumination $I$ of a point on the planet is then dependent on the power per area $P$ of the sun light and the scalar product of the sun direction $\hat{d}$ and the normal $\hat{n}$ of the surface:

$$I = P * max(\hat{n} \cdot \hat{d}, 0)$$

$P$ is proportional to the inverse square of the distance to the light source. Once again, since the planet is very far from the sun, the difference in distance between different points on the planet is negligible and $P$ is set as a constant. The illumination is then solely scaled by the scalar product. The surface normal and the sun direction are both unit vectors, and so the scalar product returns its highest value 1 when the normal is pointing towards the sun. The illumination then decreases until an angle of 90 degrees is reached where the scalar product returns 0. This becomes the rim between day and night, the so-called terminator. An angle larger than 90

degrees, meaning the surface does not receive any direct light from the sun, results in a negative scalar product. All negative values are thereby set to 0 using the max function.

The color of the pixel is then calculated through multiplication of the illumination value with the surface color. Figure 1 below displays the first rendition of this setup. The sphere, colored white, is brightest where the surface points towards the sun. The brightness then decreases until the surface becomes completely dark about halfway around the sphere, as expected.

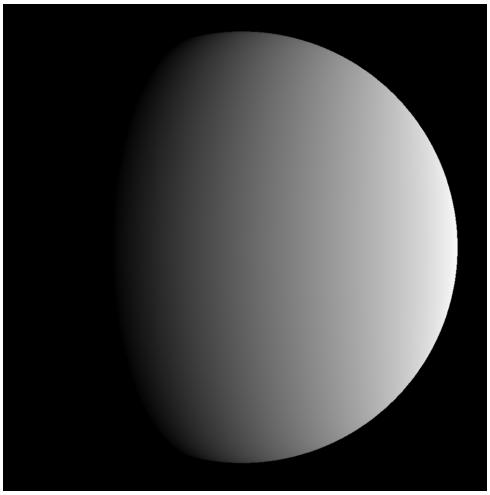


Figure 1: A smooth lit sphere rendered using ray-tracing

## 4.3 Procedural Terrain Generation

The foundation for generating the planet terrain and its normals is noise. The noise is sampled from 3D noise functions, using the intersection point of each ray as input and returning a float that can be used to determine, for example, terrain height. Noise is thereby sampled for each pixel whose ray intersects the planet surface. Sampling 3D noise by intersecting it with a sphere is inspired by a project done by youtube channel Fractal Philosophy [6]. Using this technique instead of sampling 2D noise and wrapping it around a sphere avoids the seams and polar distortion usually present with spherical UV mapping [7].

### 4.3.1 Gradient Noise

The base terrain noise is generated using a 3D gradient noise function directly based on the work of Inigo Quilez [2]. It works by dividing space into a 3D grid, where each grid corner is assigned a pseudo-random gradient vector using a hash function. For a given input sample position $\mathbf{x}$, a noise value $v_i$ is calculated for each corner $i$ using the scalar product of the corner gradient $\mathbf{g_i}$ and the distance vector from the corner position $\mathbf{p_i}$ and the sample position:

$$v_i = \mathbf{g}_i \cdot (\mathbf{x} - \mathbf{p}_i)$$

Inigo Quilez then calculates the noise value $n$ of the sample position by interpolating the corner values using quintic interpolation. This kind of interpolation blends the corner values, meaning the final noise value is expressed as a continuous analytical function. This function is differentiable, meaning the gradient of the noise function can be derived analytically during the interpolation. In his article, Inigo explains that this is faster and more accurate than calculating the gradient numerically using for example the central difference method, which involves sampling the noise multiple times. The gradient is then used to create the normals necessary for lighting the terrain.

The gradient generated by the noise function is not constrained to the sphere surface since it is the gradient of 3D noise. Only the components tangent to the sphere surface correspond to the terrain slope of the planet. The gradient $\mathbf{g}$ is therefore projected onto the tangent plane of the sphere:

$$\mathbf{g}_{surface} = \mathbf{g} - (\mathbf{g} \cdot \mathbf{s})\mathbf{s}$$

where $\mathbf{s}$ is the sphere normal. The surface gradient is then subtracted from the sphere normal, thereby tilting the normal depending on the underlying terrain. Using these terrain normals when lighting the planet, instead of the sphere normal used previously, results in the the planet shown in figure 2. This is a good showcase of the base noise, but not quite enough for an interesting planetary surface. This brings us onto the next section.

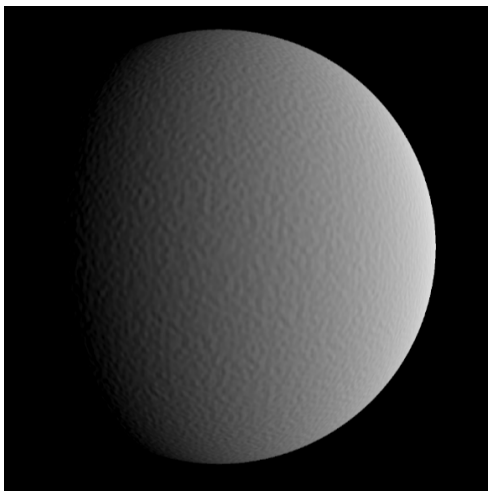


Figure 2: Terrain normals generated from gradient noise.

### 4.3.2 Fractal Brownian Motion

In another article by Inigo Quilez, he walks through a technique called Fractal Brownian Motion [3]. It can be used to generate more natural terrain with structure across multiple scales. It works by summing multiple iterations of noise, so called octaves, with simultaneously increasing frequencies and decreasing amplitudes:

$$fbm(\mathbf{x}) = \sum_{i=1}^{N} a_i \, n(f_i \mathbf{x})$$

where $N$ is the total number of octaves while $a_i$ and $f_i$ is the amplitude and frequency for the noise of octave $i$. In the implementation of fbm for this project, the frequency is doubled and the amplitude is halved for each

added octave:

$$f_{i+1} = 2f_i \qquad a_{i+1} = \frac{1}{2}a_i$$

This change of values between octaves results in fractal-like terrain where self similar structures appear on multiple scales, as can be seen below in figure 3. More octaves result in greater detail in the terrain, especially on smaller scales. This is a clear improvement on using just a single noise function, but the terrain is still fairly repetitive. This will be addressed in the next section.

Figure 3: Comparison between using 1, 2 and 10 octaves.

#### 4.3.3 Domain Warping

To reduce the repetitive appearance of the terrain, another technique described in an article by Inigo Quilez called domain warping is implemented [4]. Instead of sampling the noise directly at the input position $\mathbf{x}$, the position is first distorted using another noise function:

$$n(\mathbf{x}) = fbm(\mathbf{x} + fbm(\mathbf{x} + \mathbf{p}))$$

where the vector $\mathbf{p}$ offsets the input position. This can be used to twist and bend the noise, thereby breaking up otherwise repetitive patterns and creating more natural looking structures. The effect of domain warping can be seen below in figure 4.

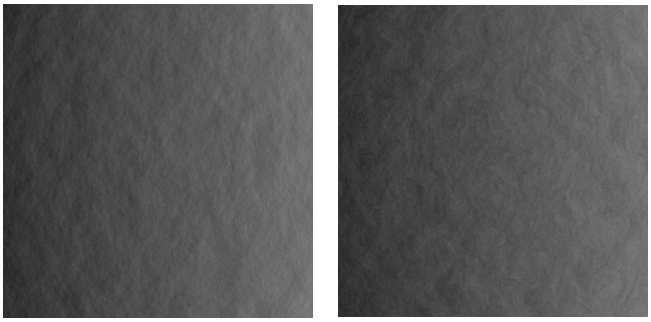


Figure 4: Comparison between with and without domain warping.

### 4.4 Surface Coloring

After the height and surface normals have been generated, this data can be used to color the planet. This section will walk through how these and other parameters are used to determine the color.

For demonstration purposes, colors imitating those of sand, rock and snow will be used to color the planet. These terrain materials are blended together using masks. A mask is a field in the range $[0, 1]$ describing how strongly a feature should appear at a given surface point. To ensure smooth transitions, the masks are generated using the smoothstep interpolation function:

$$\text{smoothstep}(a, b, x)$$

The function outputs 0 for $x$ values below $a$, 1 for values above $b$, and interpolates between the two thresholds. This is used to create a gradual transition between terrain materials.

The rock mask is based on both terrain slope and terrain height. Steeper terrain is more likely to expose rock, while higher elevations also contribute to rock formation. The slope contribution is computed as the magnitude of the surface gradient:

$$M_{\text{slope}} = ||\mathbf{g}_{\text{surface}}||$$

Flat terrain results in values close to 0 while steep terrain produces larger values. The final rock mask is then constructed by combining the slope contribution with a mask based on height:

$$M_{rock} = M_{slope} + smoothstep(h_{lower}, h_{upper}, h)$$

Beside height and slope, a temperature parameter is introduced. The temperature is calculated based on latitude as well as noise. The latitude contribution is computed using the vertical component $y$ of the sphere normal in the following manner:

$$T_{\text{lat}} = \sqrt{1 - y^2}$$

$y$ goes from 0 at the equator to $\pm 1$ at the poles, meaning $T_{lat}$ outputs its maximum temperature at the equator and its minimum temperature at the poles. The noise contribution is computed using fractal Brownian motion to introduce local climate variation. The terrain height $h(\mathbf{x})$ is subtracted from this term to produce cooler temperatures at higher elevations:

$$T_{\text{local}} = fbm(\mathbf{x}) - h(\mathbf{x})$$

The two components are then weighted and added together to form the total temperature field of the planet. Figure 5 displays a temperature field where $T_{lat}$ is weighted to be the dominant term. The temperature is at its peak at the equator and gradually decreases towards the poles as expected.

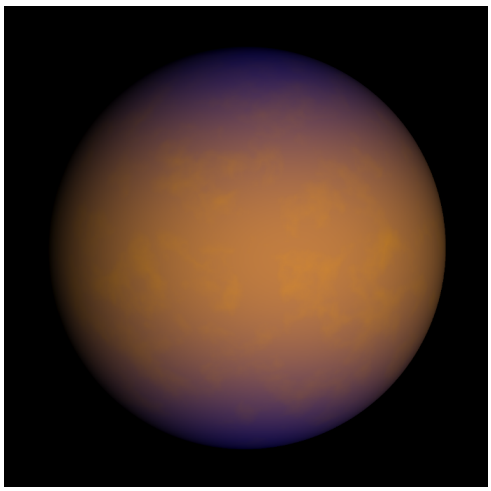


Figure 5: Visualization of the temperature field.

The temperature field $T$ is then used to determine snow coverage. A snow mask is constructed in the following way:

$$M_{\text{snow}} = 1 - \text{smoothstep}(T_{lower}, T_{upper}, T)$$

The smoothstep result is inverted in order to apply larger snow values to colder regions.

The masks are then used to compute the final surface color through successive linear interpolation. Rather than selecting a single material for each surface point, the terrain materials $\mathbf{c}_{material}$ are blended smoothly together using their corresponding masks:

$$\mathbf{c}_{\text{terrain}} = \text{lerp}\left(\text{lerp}(\mathbf{c}_{\text{sand}}, \mathbf{c}_{\text{rock}}, M_{\text{rock}}), \mathbf{c}_{\text{snow}}, M_{\text{snow}}\right)$$

where $M_{\text{rock}}$ and $M_{\text{snow}}$ determine how strongly each terrain material contributes to the final color at the current surface point. This creates smooth and natural transitions between different terrain types, as can be seen in figure 6. The poles are covered in snow while rock is exposed at steep terrain and higher elevations, and the rest is sand.

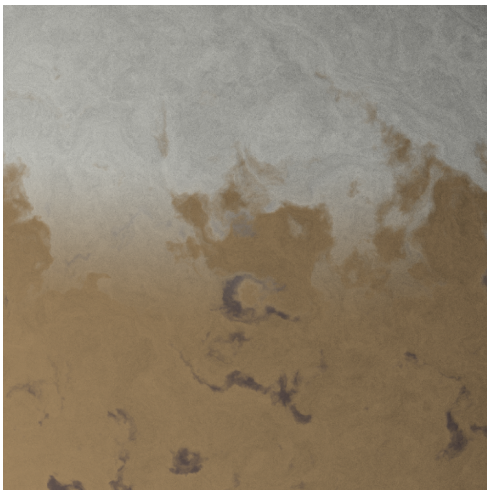


Figure 6: The planetary surface colored using masks.

This coloring setup is merely used as an example. Any other colors, parameters and masks could just as well be used to produce very different results.

### 4.5 Camera controls

Now that a planet has been generated, it is time to implement a way to explore it. This is done by making the camera controllable. The camera controls are inspired by those of tools such as Google Earth [8] and include panning as well as zoom. The camera position is represented using spherical coordinates around the planet center, controlled through yaw and pitch rotations. This locks the camera to move in orbits around the planet, keeping it pointing towards its center.

Zooming is implemented by moving the camera position closer or further away from the planet. The zoom speed is proportional to the current height above the surface to make it feel consistent regardless of zoom level. It decreases the closer the camera is, thereby allowing the camera to move towards the surface but never reach it.

The panning speed is also proportional to camera height to make panning feel consistent as well. Additional horizontal scaling is applied near the poles to compensate for the decreasing latitude circumference.

## 5 Results and Evaluation

This section will evaluate the visual results and performance of the planet renderer, as well as the strengths and limitations of the implemented techniques.

### 5.1 Visual Results

The final rendered planet can be seen below in figure 7. The combination of fractal Brownian motion and domain warping creates interesting terrain structures ranging from what resembles mountain ranges to smaller terrain details like hills and plains. Domain warping further breaks up the repetitive noise by twisting and distorting the terrain in a natural looking way. Together, these techniques produce planetary terrain that appears significantly more natural than using a single noise function alone.

The visual complexity is further increased by the rule-based coloring systems. Temperature-dependent snow coverage together with rock exposure based on slope and height create biome variation across the surface. Combined with the procedural terrain generation, these systems produce a planet that appears varied without looking completely random, instead giving the impression of terrain shaped through natural processes.

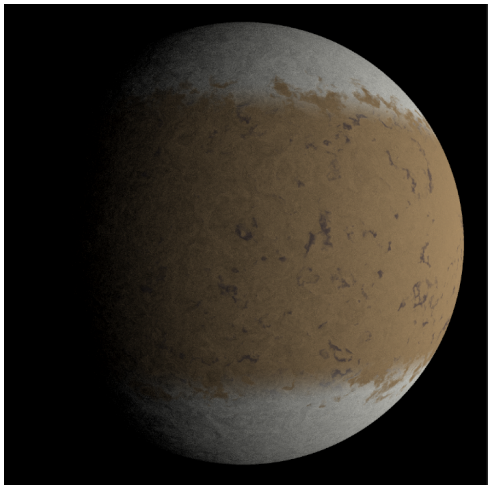


Figure 7: A full view of a planet rendered using the final rendition of the project.

Figure 8 shows how the renderer behaves across different viewing distances. This is a good demonstration of the effect of fractal Brownian motion, where detailed terrain is generated and sampled at all zoom levels due to noise functions of increasing frequency and decreasing amplitude. The analytically generated normals create a

convincing illusion of terrain height even at close viewing distances. This approximation works particularly well on a planetary scale, where terrain height is small relative to the radius of the planet.

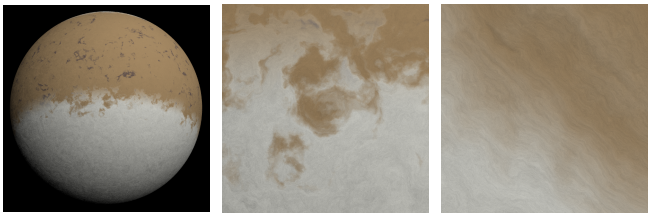


Figure 8: The same area viewed through three different zoom levels.

Figure 9 compares the terminator of a rendered planet with that of the Moon, image source LaGuardia Society of Physics Students [9]. Similar to the lunar surface, the rendered terminator creates a more dramatic display of the landscape through increased contrast between lit and shaded sides, though not quite at the same level. This demonstrates how normals alone can produce fairly convincing planetary shading while the underlying planet geometry itself remains a perfect sphere.

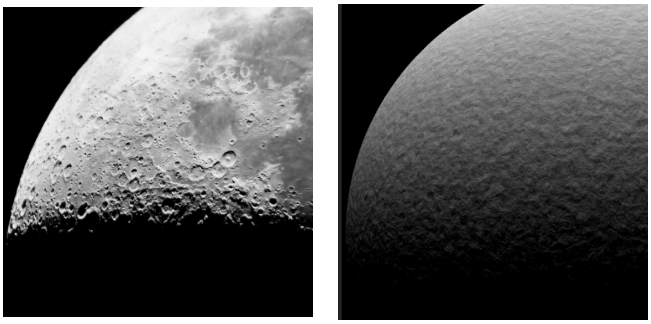


Figure 9: Comparison between the terminator of the Moon [9] and a rendered planet.

## 5.2 Limitations

The main limitation of the planet renderer is that terrain height is conveyed through normals instead of geometry displacement. This approximation works well when viewed from space, but the lack of actual displacement becomes noticeable near the planetary silhouette and at low viewing angles. This is one of the reasons for locking the camera direction towards the planet. Tilting the camera while close to the surface would lift the veil of the illusion, revealing a flat surface with a lack of mountain silhouettes.

Another consequence of using normals instead of geometry displacement is the lack of self-shadowing. Since the terrain only exists in the lighting calculations and not in the underlying geometry, mountains cannot cast shadows onto nearby terrain. This becomes especially noticeable near the terminator where the far reaching shadows visible in the reference Moon image is not quite replicated by the renderer.

Another limitation of the renderer is the use of procedural noise as the sole foundation for terrain generation. Although techniques such as fractal Brownian motion and domain warping can produce visually convincing terrain, the generated landscapes are not based on actual geological processes. The renderer therefore lacks tectonic structures, erosion, river systems and other terrain features present on real planets.

A limiting factor in this case is the fact that the entire procedural generation process is run on a GPU compute shader, where all sample points are evaluated completely independently of each other. The aforementioned processes would probably require neighboring terrain samples to influence one another, possibly over large distances, which would be significantly more difficult to parallelize on the GPU.

Another observed limitation is that increasing the number of octaves eventually produces diminishing visual returns. After roughly 20 octaves, additional terrain detail become less noticeable even when zooming in further. Somehow the contribution of the very high frequencies are suppressed, possibly during interpolation or due to numerical precision. The reason for this is currently unclear and will require further investigation.

## 5.3 Performance

The performance of the renderer is primarily dependent on three factors: render resolution, how much of the screen is covered by the planet, and the number of octaves used during procedural terrain generation.

The render resolution determines the number of pixels that need to be processed each frame. Since each pixel independently casts a ray, computes a sphere intersection, samples procedural noise and evaluates lighting, increasing the resolution increases the computational cost approximately linearly with the number of pixels. The performance cost of render resolution was measured while keeping the octave count fixed and letting the planet cover the entire screen. The results can be seen in Table 1.

| Resolution | FPS |
|---|---|
| $256 \times 256$ | 800 |
| $512 \times 512$ | 700 |
| $1024 \times 1024$ | 330 |
| $2048 \times 2048$ | 100 |

Table 1: Performance measurements for different render resolutions using a fixed octave count of 10 with the planet covering the entire screen.

Performance is also strongly dependent on how much of the screen the planet occupies. Pixels that do not intersect the planet will not sample it, thereby contributing less to the total computational cost of the renderer. Rendering the planet up close is therefore more expensive than viewing it from far away. However, once the planet covers the entire screen, zooming in further does not affect the performance since the amount of planet samples stay unchanged.

The final major performance factor is the number of

octaves used in the fractal Brownian motion calculations, since each additional octave requires another noise evaluation. Higher octave counts therefore increase the terrain detail while simultaneously increasing the computational cost of the renderer. The performance cost of increasing the number of octaves was measured at a fixed render resolution. The results are shown in Table 2.

| Octaves | FPS |
|---|---|
| 1 | 400 |
| 5 | 370 |
| 10 | 330 |
| 20 | 270 |

Table 2: Performance measurements for different octave counts at a fixed resolution of $1024 \times 1024$ with the planet covering the entire screen.

The performance measurements were performed on a system with an NVIDIA RTX 3060 Ti GPU and an INTEL i7 13700k CPU.

# 6 Future Work

There is room for improvement of the procedural generation implemented in this project. One possibility would be to introduce additional terrain parameters, such as humidity and geological factors, together with more complex rules dictating their interactions. More parameters and interactions between terrain features could further increase the visual complexity and realism of the generated planets.

Atmospheric rendering would be a significant addition to the planet renderer. Atmospheric scattering would greatly improve the visual appearance of the planet and add much to its realism. Clouds, rendered either as another layer of projected 3D noise on top of the terrain or volumetric in the atmosphere, could also improve the the visual quality of the renderer.

Finally, the current terrain representation could possibly be extended through raymarched geometric displacement. Instead of relying solely on normals, the ray intersection itself could be modified to take the procedurally generated height into account, allowing the terrain to exist as geometry. This would enable mountain silhouettes and self-shadowing, thereby addressing several of the main limitations of the current renderer. The performance of such a setup is however unclear.

# 7 Conclusion

This project demonstrates how procedural noise techniques together with ray-traced rendering can be combined to produce visually convincing planetary rendering in real time using GPU compute shaders. The project also highlights the limitations of relying solely on normals instead of displaced geometry, particularly regarding silhouettes and self-shadowing. Overall, the project serves as a strong foundation for future expansions such as atmospheric rendering, clouds and raymarched terrain displacement.

# References


1. Hello games, "No Man's Sky," *Wikipedia*. Available: `https://en.wikipedia.org/wiki/No_Man%27s_Sky`
2. Inigo Quilez, "Gradient Noise," *iquilezles.org*. Available: `https://iquilezles.org/articles/gradientnoise/`
3. Inigo Quilez, "Fractal Brownian Motion," *iquilezles.org*. Available: `https://iquilezles.org/articles/fbm/`
4. Inigo Quilez, "Warping," *iquilezles.org*. Available: `https://iquilezles.org/articles/warp/`
5. DH2323 Computer Graphics and Interaction, "Rendering Track Lab 2," course material, KTH Royal Institute of Technology, 2026.
6. Fractal Philosophy, "Procedural Planet Rendering," YouTube video. Available: `https://www.youtube.com/watch?v=7xL0udlhnqI`
7. "UV Mapping," *Wikipedia*. Available: `https://en.wikipedia.org/wiki/UV_mapping`
8. Google Earth. Available: `https://earth.google.com/web/`
9. LaGuardia Society of Physics Students, "The Moon Terminator," *Ad Astra*. Available: `http://www.adastraletter.com/2021/3/the-moon-terminator`